Squares of a Sorted Array

Two approaches, visualized execution, and the core logic to remember

Brute Force · O(n log n) Two Pointers · O(n)

Problem Statement

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number, also sorted in non-decreasing order.

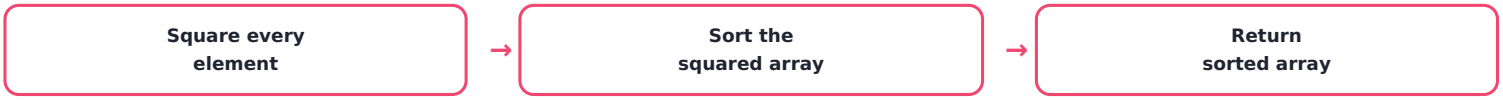
Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	O(n log n)	O(n)	Square everything, then sort
Two Pointers	O(n)	O(n)	Sorted input → biggest square is always at an end

Big picture: The array is already sorted — the brute force approach throws that fact away and re-sorts from scratch. The two-pointer approach exploits it: the largest square always sits at one of the two ends of the original array, so you can build the result directly without a full sort.

1 · Brute Force

O(n log n) time **O(n) space**

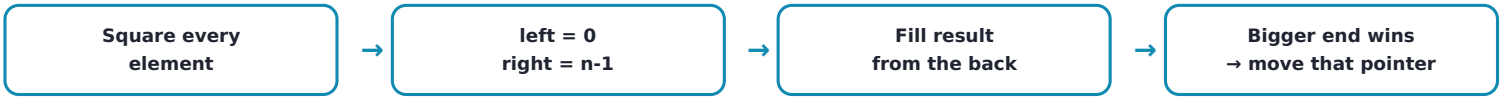



```java
// square every element, then sort
private static int[] bruteForceSolution(int[] nums) {
    int[] sqArray = Arrays.stream(nums).map(s -> s * s).toArray();
    Arrays.sort(sqArray);
    return sqArray;
}
```

Recall trick: "Square first, sort after." Correct and simple, but it ignores the fact that `nums` is already sorted — that's free information being wasted.

2 · Two Pointers

O(n) time O(n) space




```java
// sorted input: the biggest square is always at one of the two ends
private static int[] usingTwoPointers(int[] nums) {
    int[] squareArray = Arrays.stream(nums).map(s -> s * s).toArray();
    int[] res = new int[nums.length];
    int left = 0;
    int right = nums.length - 1;

    for (int i = squareArray.length - 1; i >= 0; i--) {
        if (squareArray[left] >= squareArray[right]) {
            res[i] = squareArray[left];
            left++;
        } else {
            res[i] = squareArray[right];
            right--;
        }
    }
    return res;
}
```

Trace for `nums = [-4,-1,0,3,10]` (`squareArray = [16,1,0,9,100]`):

i	left	right	squareArray[left]	squareArray[right]	picked	next
4	0	4	16	100	100 → res[4]	right--
3	0	3	16	9	16 → res[3]	left++
2	1	3	1	9	9 → res[2]	right--
1	1	2	1	0	1 → res[1]	left++
0	**2**	**2**	**0**	**0**	**0 → res[0]**	**done → [0,1,9,16,100]**

Recall trick: "Sorted array, fill result from the back, always take the bigger end." Since squaring is monotonic on each side of zero, whichever end has the bigger absolute value produces the bigger square — no need to look at signs, just compare the squares directly.

TL;DR — For Future Me

What to remember

- **Best solution:** Two pointers, O(n) — "fill from the back, always take the bigger of the two ends."
- **Brute force** is correct but wastes the fact that the input is already sorted.
- **Underlying pattern:** whenever an array is sorted and you need extremes (largest, smallest, or here — largest square), think two pointers converging from both ends before reaching for a full sort.